

Guía de laboratorio: Dockerfile y Docker Compose con Spring Boot + MySQL

Contexto del laboratorio

Esta guía enseña a empaquetar una aplicación Java Spring Boot en un contenedor Docker y a levantarla junto con una base de datos MySQL usando Docker Compose.

La temática del ejemplo es un **catálogo de cartas**, porque permite entender mejor la idea de guardar datos en una base de datos y exponerlos desde una aplicación web simple.

Objetivo general

Al finalizar este laboratorio, el estudiante será capaz de construir una imagen Docker para una aplicación Spring Boot, ejecutar un contenedor a partir de esa imagen y levantar un entorno completo con aplicación más base de datos usando `docker compose`.

Objetivos específicos

- Identificar para qué sirve un `Dockerfile` en una aplicación Java.
- Comprender la diferencia entre **imagen** y **contenedor**.
- Construir una imagen Docker que ejecute un archivo JAR de Spring Boot.
- Configurar una conexión entre Spring Boot y MySQL dentro de Docker Compose usando el nombre del servicio como host.
- Ejecutar y detener varios contenedores con comandos simples.

Idea explicada con lenguaje simple

Piensa en la aplicación como un **juego de cartas** y en la base de datos como una **caja donde guardas la colección**. Docker permite meter la aplicación en una caja cerrada que ya trae Java instalado, de modo que pueda ejecutarse igual en cualquier computador.

El archivo `Dockerfile` es la receta para construir esa caja. El archivo `docker-compose.yml` es el tablero que dice qué cajas se van a levantar y cómo se conectan entre sí, por ejemplo, una caja para la app y otra para MySQL.

Requisitos previos

Antes de comenzar, el estudiante debe tener instalado Docker Desktop o Docker Engine con Docker Compose habilitado, además de Java y Maven para compilar el proyecto Spring Boot.

También debe contar con un proyecto Spring Boot funcional que pueda empaquetarse como archivo JAR, aunque el código Java del ejemplo puede ser reemplazado por cualquier aplicación simple que ya exista.

Estructura esperada del proyecto

La carpeta del proyecto debería verse de forma similar a esta:

```
mi-proyecto-cartas/  
├── src/  
├── target/  
├── pom.xml  
├── Dockerfile  
└── docker-compose.yml
```

La carpeta `target/` aparece después de ejecutar `mvn clean package`, porque ahí Maven deja el JAR compilado que luego Docker copiará dentro de la imagen.

Caso de uso del ejemplo

La aplicación representa un catálogo de cartas. Cada carta podría tener campos como `id`, `nombre`, `tipo`, `rareza` y `ataque`, y esos datos se almacenan en MySQL.

Ejemplos de endpoints que el proyecto podría tener:

- `GET /cartas` para listar cartas.
- `GET /cartas/{id}` para buscar una carta.
- `POST /cartas` para guardar una carta nueva.

Este patrón encaja muy bien con ejemplos típicos de Spring Boot conectados a MySQL en Docker Compose.

Parte 1: Configurar la conexión a MySQL

En el archivo `src/main/resources/application.properties`, la aplicación debe quedar configurada para conectarse a MySQL usando variables de entorno.

```
spring.datasource.url=jdbc:mysql://db:3306/cartasdb?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Explicación importante

- `jdbc:mysql://db:3306/cartasdb...` indica que la base de datos está en el host `db` y en el puerto `3306`.
- `db` no es `localhost`; `db` será el nombre del servicio de MySQL dentro de `docker-compose.yml`.
- `DB_USERNAME` y `DB_PASSWORD` se entregarán más adelante desde Docker Compose.

Parte 2: Crear el Dockerfile

Crear un archivo llamado `Dockerfile` en la raíz del proyecto, al mismo nivel que `pom.xml`.

Contenido sugerido:

```
FROM openjdk:17-jdk-alpine

WORKDIR /app

COPY target/cartas-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Qué hace cada línea

Línea	Explicación
<code>FROM openjdk:17-jdk-alpine</code>	Usa una imagen base que ya trae Java 17 instalado.
<code>WORKDIR /app</code>	Define la carpeta de trabajo dentro del contenedor.
<code>COPY target/cartas-0.0.1-SNAPSHOT.jar app.jar</code>	Copia el archivo JAR compilado desde la carpeta <code>target</code> al contenedor.
<code>EXPOSE 8080</code>	Declara que la aplicación escucha en el puerto 8080.
<code>ENTRYPOINT ["java", "-jar", "app.jar"]</code>	Indica el comando con que se inicia la aplicación.

Atención

El nombre `cartas-0.0.1-SNAPSHOT.jar` es solo un ejemplo. Cada estudiante debe reemplazarlo por el nombre real del JAR generado en la carpeta `target`.

Parte 3: Compilar la aplicación

Desde la raíz del proyecto, ejecutar:

```
mvn clean package
```

Este comando compila el proyecto y genera el archivo JAR dentro de la carpeta `target`, que luego será usado por Docker para construir la imagen.

Verificación

Revisar que exista un archivo similar a este:

```
target/cartas-0.0.1-SNAPSHOT.jar
```

Si el archivo no existe, el `docker build` fallará porque el `COPY` del Dockerfile no encontrará nada para copiar.

Parte 4: Construir la imagen Docker

Una vez generado el JAR, construir la imagen con este comando:

```
docker build -t cartas-app:1.0 .
```

Explicación del comando

- `docker build`: construye una imagen a partir del Dockerfile.
- `-t cartas-app:1.0`: asigna nombre `cartas-app` y etiqueta `1.0`.
- `.`: indica que el contexto de construcción es la carpeta actual.

Verificar imágenes disponibles

```
docker images
```

La imagen `cartas-app` debería aparecer en la lista si el proceso terminó correctamente.

Parte 5: Probar la aplicación sola en un contenedor

Ejecutar la imagen recién creada:

```
docker run --name cartas-demo -p 8080:8080 cartas-app:1.0
```

Explicación del comando

- `--name cartas-demo`: asigna nombre al contenedor.
- `-p 8080:8080`: publica el puerto 8080 del contenedor en el puerto 8080 del computador.
- `cartas-app:1.0`: usa la imagen construida anteriormente.

Qué probar

Abrir navegador, Postman o `curl` y consultar un endpoint de la aplicación, por ejemplo:

```
curl http://localhost:8080/cartas
```

Si la app no depende aún de MySQL para responder este endpoint, debería contestar desde el contenedor sin problemas.

Detener y eliminar el contenedor

```
docker stop cartas-demo  
docker rm cartas-demo
```

Parte 6: Crear docker-compose.yml

Ahora se levantará la aplicación junto con MySQL usando un solo archivo.

Crear `docker-compose.yml` en la raíz del proyecto con este contenido:

```
version: "3.8"  
  
services:  
  db:  
    image: mysql:8.0  
    container_name: cartas-mysql  
    environment:  
      MYSQL_ROOT_PASSWORD: root  
      MYSQL_DATABASE: cartasdb
```

```
  MYSQL_USER: cartas
  MYSQL_PASSWORD: cartas123
  ports:
    - "3306:3306"
  volumes:
    - cartas-data:/var/lib/mysql

app:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: cartas-app
  ports:
    - "8080:8080"
  environment:
    DB_USERNAME: cartas
    DB_PASSWORD: cartas123
  depends_on:
    - db

volumes:
  cartas-data:
```

Parte 7: Entender docker-compose.yml

Servicio db

El servicio db usa la imagen oficial mysql:8.0, que trae MySQL listo para ejecutarse sin instalarlo manualmente en el computador.

Las variables MYSQL_DATABASE, MYSQL_USER y MYSQL_PASSWORD permiten crear automáticamente una base de datos y un usuario cuando el contenedor inicia por primera vez.

La línea volumes: - cartas-data:/var/lib/mysql guarda los datos de MySQL en un volumen, de modo que no se pierdan al detener o borrar el contenedor.

Servicio app

El servicio app no usa una imagen descargada, sino que la construye con el Dockerfile del proyecto usando build.context y dockerfile.

Las variables `DB_USERNAME` y `DB_PASSWORD` coinciden con las usadas en `application.properties`, lo que permite que Spring Boot reciba esos valores al iniciar.

`depends_on` indica que `app` depende de `db`, por lo que Compose intentará iniciar primero el contenedor de MySQL.

Parte 8: Levantar todo el entorno

Ejecutar:

```
docker compose up --build
```

Qué hace este comando

- Construye la imagen de la aplicación si es necesario.
- Levanta el contenedor de MySQL.
- Levanta el contenedor de la aplicación.
- Muestra los logs de ambos servicios en la misma consola.

El parámetro `--build` es útil cuando hubo cambios en el código fuente o en el Dockerfile, porque obliga a reconstruir la imagen antes de iniciar los contenedores.

Parte 9: Verificar que todo funciona

En otra terminal, ejecutar:

```
docker compose ps
```

Si todo salió bien, deberían aparecer dos servicios activos: `cartas-mysql` y `cartas-app`.

Luego probar la aplicación:

```
curl http://localhost:8080/cartas
```

Si la aplicación inserta o consulta datos, esos datos deberían quedar persistidos en MySQL gracias al volumen configurado en Compose.

Parte 10: Detener el entorno

Para detener todos los contenedores definidos en el archivo Compose:

```
docker compose down
```

Para detenerlos y además borrar el volumen de datos:

```
docker compose down -v
```

Usar `-v` elimina también el volumen llamado `cartas-data`, por lo que se perderán los registros guardados en MySQL.

Diagrama mental del laboratorio

```
+-----+ se conecta a +-----+
| Contenedor app | -----> | Contenedor MySQL |
| Spring Boot    |          | Base de datos cartas |
| Puerto 8080    |          | Puerto 3306      |
+-----+          +-----+
```

La aplicación no se conecta a `localhost`, sino al servicio `db`, porque ambos contenedores comparten la red interna creada por Docker Compose.

Actividad guiada

Actividad 1: Preparar el proyecto

1. Abrir el proyecto Spring Boot.
2. Revisar `application.properties`.
3. Confirmar que el nombre del JAR que se genera coincida con el usado en el Dockerfile.

Actividad 2: Construir y probar la app sola

1. Ejecutar `mvn clean package`.
2. Ejecutar `docker build -t cartas-app:1.0 ..`
3. Ejecutar `docker run --name cartas-demo -p 8080:8080 cartas-app:1.0`.
4. Probar un endpoint.
5. Detener y borrar el contenedor.

Actividad 3: Trabajar con Compose

1. Crear `docker-compose.yml`.
2. Ejecutar `docker compose up --build`.
3. Revisar `docker compose ps`.
4. Probar el endpoint nuevamente.
5. Detener el entorno con `docker compose down`.

Errores comunes y cómo solucionarlos

Error	Causa probable	Solución
COPY failed al hacer build	El JAR no existe o el nombre del archivo en el Dockerfile está mal.	Ejecutar <code>mvn clean package</code> y revisar el nombre real en <code>target/</code> .
La app no conecta a MySQL	La URL JDBC usa localhost en vez de db.	Cambiar el host a db en <code>application.properties</code> .
La app no levanta con cambios recientes	Se usó <code>docker compose up</code> sin reconstruir.	Ejecutar <code>docker compose up --build</code> .
El puerto 8080 está ocupado	Ya existe otra app usando ese puerto.	Cambiar el mapeo a otro, por ejemplo <code>8081:8080</code> .
Se perdieron los datos	Se ejecutó <code>docker compose down -v</code> .	No usar <code>-v</code> si se desea conservar la información.

Preguntas de reflexión

1. ¿Cuál es la diferencia entre una imagen y un contenedor?
2. ¿Por qué dentro de Docker Compose la aplicación no debe usar localhost para conectarse a MySQL?
3. ¿Qué ventaja entrega el volumen `cartas-data` en este laboratorio?
4. ¿Qué pasa si cambia el código fuente y se ejecuta `docker compose up` sin `--build`?
5. ¿Qué problema resuelve Docker al trabajar en distintos computadores?

Hoja rápida de comandos

Acción	Comando
Compilar proyecto	<code>mvn clean package</code>
Construir imagen	<code>docker build -t cartas-app:1.0 .</code>
Ejecutar app sola	<code>docker run --name cartas-demo -p 8080:8080 cartas-app:1.0</code>
Ver imágenes	<code>docker images</code>
Detener contenedor	<code>docker stop cartas-demo</code>
Eliminar contenedor	<code>docker rm cartas-demo</code>
Levantar entorno completo	<code>docker compose up --build</code>
Ver servicios	<code>docker compose ps</code>
Bajar entorno	<code>docker compose down</code>
Bajar entorno y borrar volumen	<code>docker compose down -v</code>

Cierre didáctico

Este laboratorio muestra que Docker no reemplaza el código Java ni la base de datos, sino que ayuda a empaquetarlos y ejecutarlos de forma consistente. Spring Boot sigue siendo la aplicación, MySQL sigue siendo la base de datos y Docker solo organiza cómo se construyen y cómo se levantan.

La gran idea que el estudiante debe recordar es esta: **una app puede vivir en un contenedor, la base de datos en otro, y Docker Compose permite levantarlos juntos y conectados con pocos comandos.**